

DESIGN AND VERIFICATION OF A 32-BIT ALU WITH STATUS FLAGS USING VERILOG HDL

Submitted by:

CHANDANA SRI SUNKARA

Electronics and Communication Engineering

Rajiv Gandhi University of Knowledge
Technologies

Abstract

This project presents the design and verification of a 32-bit Arithmetic Logic Unit (ALU) using Verilog HDL. The ALU performs arithmetic, logical, shift, and comparison operations based on a 4-bit opcode. Status flags such as Zero, Carry, Overflow, and Negative are generated to indicate the result condition. Functional verification was carried out using a Verilog testbench and Vivado simulation environment.

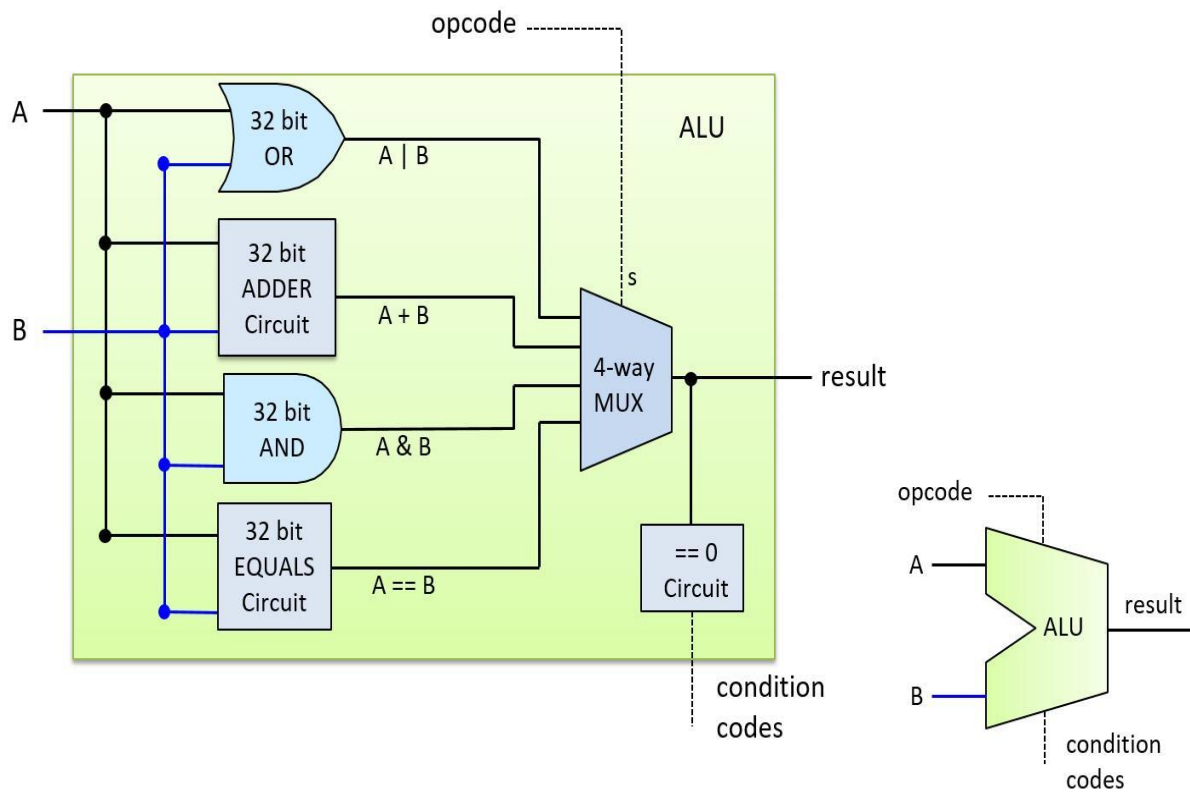
Introduction

An Arithmetic Logic Unit (ALU) is a fundamental building block of digital processors responsible for performing arithmetic and logical operations. Modern processors rely on ALUs to execute mathematical calculations, logical decisions, comparisons, and data manipulation operations. This project implements a 32-bit ALU supporting multiple operations and verifies its functionality through simulation.

Objectives

- * Design a 32-bit ALU using Verilog HDL.
- * Implement arithmetic, logical, shift, and comparison operations.
- * Generate status flags.
- * Verify functionality using simulation.
- * Analyse RTL architecture and simulation waveforms.

ALU Architecture



Inputs:

- A [31:0]
- B [31:0]
- Opcode [3:0]

Outputs:

- Result [31:0]
- zero_flag
- carry_flag
- overflow_flag
- negative_flag

Opcodes:

0000 – ADD

0001 – SUB
0010 – AND
0011 – OR
0100 – XOR
0101 – NAND
0110 – NOR
0111 – XNOR
1000 – LEFT SHIFT
1001 – RIGHT SHIFT
1010 – EQUAL
1011 – GREATER THAN
1100 – NOT
1101 – LESS THAN

Flag Description

Zero Flag: Set when result equals zero.

Carry Flag: Indicates carry generation during arithmetic operations.

Overflow Flag: Indicates signed arithmetic overflow.

Negative Flag: Represents the sign of the result using the MSB.

RTL Design

```
module alu_32bit (  
    input [31:0] A,  
    input [31:0] B,  
    input [3:0] opcode,
```

```

output reg [31:0] result,
output reg zero_flag,
output reg carry_flag,
output reg overflow_flag,
output reg negative_flag
);
reg [32:0] temp;
always @ (*) begin
    // Default values
    result = 32'b0;
    carry_flag = 1'b0;
    overflow_flag = 1'b0;
    case(opcode)
        // ADD
        4'b0000: begin
            temp = A + B;
            result = temp [31:0];
            carry_flag = temp [32];
            overflow_flag = (~A [31] & ~B [31] & result [31]) | (A
[31] & B [31] & ~result [31]);
        end
        // SUB
        4'b0001: begin
            temp = A - B;
            result = temp [31:0];
            overflow_flag = (~A [31] & B [31] & result [31]) | (A
[31] & ~B [31] & ~result [31]);
        end
        // AND
        4'b0010: begin
            result = A & B;

```

```
end
// OR
4'b0011: begin
    result = A | B;
end
// XOR
4'b0100: begin
    result = A ^ B;
end
// NAND
4'b0101: begin
    result = ~ (A & B);
end
// NOR
4'b0110: begin
    result = ~ (A | B);
end
// XNOR
4'b0111: begin
    result = ~ (A ^ B);
end
// Left Shift
4'b1000: begin
    result = A << 1;
end
// Right Shift
4'b1001: begin
    result = A >> 1;
end
// Equal
4'b1010: begin
```

```

        result = (A == B)? 32'd1: 32'd0;
    end
    // Greater Than
    4'b1011: begin
        result = (A > B)? 32'd1: 32'd0;
    end
    default: begin
        result = 32'd0;
    end
endcase
// Flag generation
zero_flag = (result == 0);
negative_flag = result [31];
end
endmodule

```

Verification Methodology

A Verilog testbench was developed to verify all supported operations and status flags. Directed test vectors were applied to validate arithmetic, logical, shift, comparison, and corner-case conditions.

```

module alu_32bit_tb;
reg [31:0] A;
reg [31:0] B;
reg [3:0] opcode;
wire [31:0] result;
wire zero_flag;
wire carry_flag;
wire overflow_flag;
wire negative_flag;

```

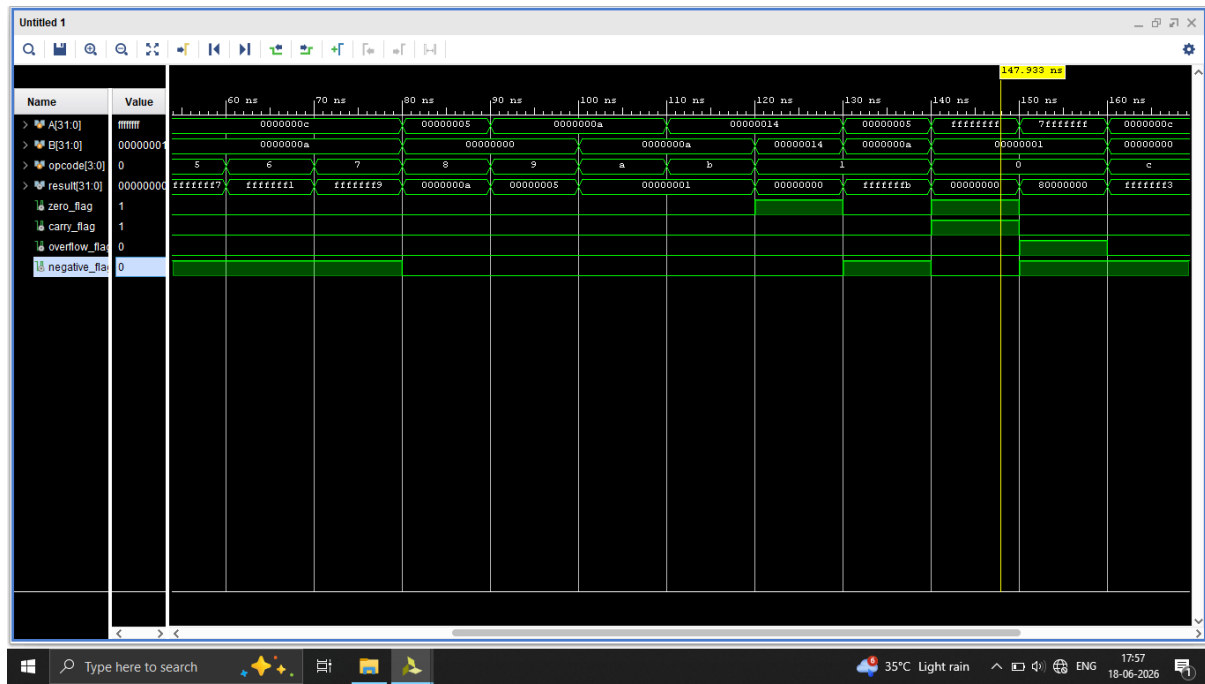
```
alu_32bit DUT (.A(A),
    .B(B),
    .opcode(opcode),
    .result(result),
    .zero_flag(zero_flag),
    .carry_flag(carry_flag),
    .overflow_flag(overflow_flag),
    .negative_flag(negative_flag) );
initial begin
    // ADD
    A = 20;
    B = 10;
    opcode = 4'b0000;
    #10;
    // SUB
    A = 20;
    B = 10;
    opcode = 4'b0001;
    #10;
    // AND
    A = 12;
    B = 10;
    opcode = 4'b0010;
    #10;
    // OR
    A = 12;
    B = 10;
    opcode = 4'b0011;
    #10;
    // XOR
    A = 12;
```



```
B = 10;
opcode = 4'b0100;
#10;
// NAND
A = 12;
B = 10;
opcode = 4'b0101;
#10;
// NOR
A = 12;
B = 10;
opcode = 4'b0110;
#10;
// XNOR
A = 12;
B = 10;
opcode = 4'b0111;
#10;
// LEFT SHIFT
A = 5;
B = 0;
opcode = 4'b1000;
#10;
// RIGHT SHIFT
A = 10;
B = 0;
opcode = 4'b1001;
#10;
// EQUAL
A = 10;
B = 10;
```

```
opcode = 4'b1010;
#10;
// GREATER THAN
A = 20;
B = 10;
opcode = 4'b1011;
#10;
// ZERO FLAG TEST
A = 20;
B = 20;
opcode = 4'b0001;
#10;
// NEGATIVE FLAG TEST
A = 5;
B = 10;
opcode = 4'b0001;
#10;
// CARRY FLAG TEST
A = 32'hFFFFFFFF;
B = 32'h00000001;
opcode = 4'b0000;
#10;
// OVERFLOW FLAG TEST
A = 32'h7FFFFFFF;
B = 32'h00000001;
opcode = 4'b0000;
#10;
$finish;
end
endmodule
```

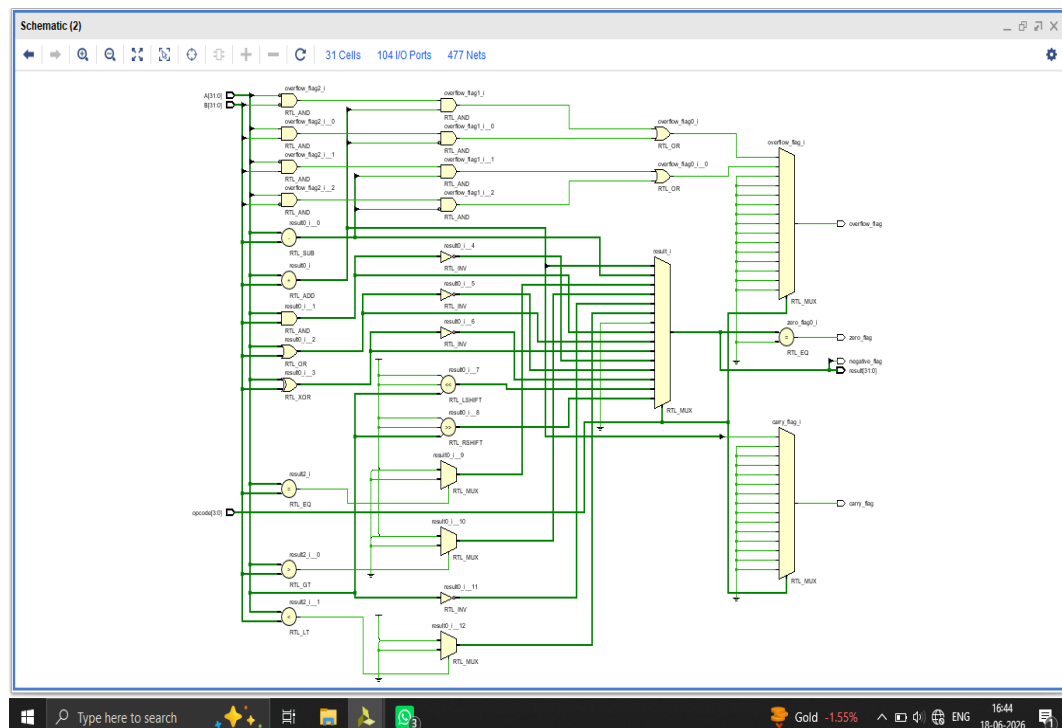
Simulations-Waveforms



Observations:

- All operations produced expected outputs.
- Flags were generated correctly.
- Corner cases were successfully verified.

RTL Schematic



The schematic confirms the multiplexer-based architecture and flag generation logic inferred by the synthesis tool.

Results: The designed ALU successfully performed all arithmetic, logical, shift, and comparison operations. Verification results confirmed correct functionality and proper status flag generation.

Conclusion: A 32-bit ALU was successfully designed and verified using Verilog HDL. The design supports multiple operations and generates status flags required in processor architectures. Simulation and RTL analysis verified the correctness of the implementation.

Future Scope:

- Self-checking testbench
- Randomized verification
- SystemVerilog implementation
- UVM-based verification
- Integration into a processor datapath

References:

- Verilog HDL by Samir Palnitkar
- Digital Logic Design Concepts